

Serie 12

Aufgabe 1:

Lösen Sie das folgende Gleichungssystem mithilfe von `numpy`, einmal direkt und einmal indem Sie die Inverse berechnen. Machen Sie die Probe. Was beobachten Sie?

$$\begin{aligned}x + y + 2z &= 3 \\ 2x + 3y + z &= 5 \\ 7x + 9y - 3z &= 0\end{aligned}$$

Aufgabe 2:

Finden Sie den Aufwand von Matrix-Matrix-Multiplikation `@` empirisch heraus, indem Sie die Zeit messen, die für die Berechnung von `A @ B` benötigt wird, wobei A und B zufällige $n \times n$ -Matrizen sind. Variieren Sie n und erstellen Sie einen `log-log`-Plot der benötigten Zeit in Abhängigkeit von n . Welcher Zusammenhang lässt sich erkennen?

Aufgabe 3:

Der Strassen-Algorithmus ist ein schneller Algorithmus zur Matrix-Matrix-Multiplikation. Der Algorithmus ist rekursiv definiert: Wir nehmen der Einfachheit halber an, dass die Matrizen A und B quadratisch sind und eine Größe von $n \times n$ haben, wobei n eine Potenz von 2 ist, also $n = 2^k$ für ein $k \in \mathbb{N}$.

1. Falls die Matrizen A und B eine Größe kleiner oder gleich 2×2 haben, berechne das Produkt $C = A @ B$ direkt mit der Standard-Methode.
2. Zerlege die beiden $n \times n$ -Matrizen A und B in vier $n/2 \times n/2$ -Untermatrizen.

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

3. Berechne die folgenden sieben Hilfsmatrizen (jede Matrix-Matrix-Multiplikation wird rekursiv wieder mit dem Strassen-Algorithmus durchgeführt):

$$\begin{aligned}M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\ M_2 &= (A_{21} + A_{22})B_{11} \\ M_3 &= A_{11}(B_{12} - B_{22}) \\ M_4 &= A_{22}(B_{21} - B_{11}) \\ M_5 &= (A_{11} + A_{12})B_{22} \\ M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}) \\ M_7 &= (A_{12} - A_{22})(B_{21} + B_{22})\end{aligned}$$

4. Setze die Untermatrizen der Ergebnis-Matrix $C = AB$ wie folgt zusammen:

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

Implementieren Sie den Strassen-Algorithmus und zeigen Sie wie in Aufgabe 2, dass der Algorithmus den Aufwand von $O(n^{\log_2 7}) \approx O(n^{2.81})$ hat.

Aufgabe 4

Gegeben sei die Tridiagonalmatrix $A = \begin{pmatrix} 4 & -1 & & \\ -1 & 4 & -1 & \\ & -1 & 4 & \ddots \\ & & \ddots & \ddots \end{pmatrix} \in \mathbb{R}^{n \times n}$. Erstellen

Sie mithilfe von `numpy` eine Blockmatrix der Form

$$C = \begin{pmatrix} A & -I & & \\ -I & A & -I & \\ & -I & A & \ddots \\ & & \ddots & \ddots \end{pmatrix} \in \mathbb{R}^{n^2 \times n^2}.$$

Aufgabe 5:

In der Datei `vertices.npy` sind die Eckpunkte eines Polygons gespeichert. Laden Sie die Datei, und benutzen Sie die **Monte-Carlo** Methode, um die Fläche des Polygons zu berechnen. Benutzen Sie dabei die Strategie aus der Vorlesung, wo die Fläche von Österreich berechnet wird. Visualisieren Sie Ihre Berechnung mithilfe von `matplotlib`.

Aufgabe 6:

Schreiben Sie eine Python-Funktion, die eine Funktion $f : [a, b] \rightarrow \mathbb{R}$, die Intervallgrenzen a und b und eine Anzahl n entgegennimmt und das Integral

$$\int_a^b f(x) dx$$

mithilfe der Monte-Carlo Methode approximiert. Dazu erzeugen Sie n zufällige Punkte im Intervall $[a, b]$ mit `numpy.random.uniform` und berechnen den Mittelwert der Funktionswerte an diesen Punkten. Multiplizieren Sie den Mittelwert mit der Länge des Intervalls $(b - a)$, um die Approximation des Integrals zu erhalten.

Wählen Sie eine Funktion deren Integral sie exakt kennen und erstellen Sie einen **log-log** -Plot des Fehlers der Monte-Carlo Approximation in Abhängigkeit von n . Da das Ergebnis vom Zufall abhängt, machen Sie für jeden Wert von n mehrere Berechnungen und mitteln Sie den Fehler über diese Berechnungen. Sie sollten beobachten, dass der Fehler ungefähr proportional zu $1/\sqrt{n}$ ist.